Week 4: Building the Backend: Express 2

Problem Statement

This week we continue building the backend for **ThreadHive**, a social media platform where users can create communities (subreddits), post discussion topics (threads), add comments, vote and interact with content.

Your job is to build the **REST API layer** that powers this platform using **Node.js**, **Express.js**, and **MongoDB (Mongoose)**.

Project Setup and Structure

1. Download and extract the assignment starter code from the zip file provided on Olympus.
2. Initialize a new Git repository (either by running `git init` in the terminal or by using the VS Code Git extension).
3. As you make changes throughout this tutorial, remember to regularly commit your work.

The starter code shared follows a clean layered architecture:

```
src/
├── models/        → Mongoose schemas (User, Subreddit, Thread, Comment)
├── services/      → Database logic (queries, mutations)
├── controllers/   → Request/response handling
├── middleware/    → Auth and error handling middleware
└── routes/        → Express route definitions
```

We have already built these layers for the Subreddit and Thread api endpoints last week. The starter code additionally includes functionality for the Comments and Votes. This week you will implement the authentication services and middleware, secure the backend, write tests and document the codebase. We will do this in two parts:

	Part	Focus	AI Usage
Part 1	Manual Implementation	Authentication Services (register, login), Authentication Middleware	No AI Tools
Part 2	AI-Assisted Development	Security Enhancements, Unit and Integration Testing, Documentation	GitHub Copilot

In **Part 1** you will build foundational understanding by writing every line of code yourself. In **Part 2** we will use GitHub Copilot workflows to see how AI tools can help accelerate the development process.

By the end of this session, you will be able to:

- Build a secure authentication system for a Node.js backend
- Use AI tools to identify and fix security vulnerabilities in your code
- Create a custom agent to generate unit and integration tests for your API endpoints
- Build a custom agent to generate a comprehensive README file for your project

---

Part 1: Manual Implementation — Authentication and Error Handling

In this part, you will implement the entire authentication system for the backend manually, without using any AI tools. This will help you build a strong foundational understanding of how authentication works in a Node.js backend. You will also add error handling to services already implemented in previous weeks.

1. Start by adding error handling to the existing services in `services/threadService.js` and `services/subredditService.js`. Review and use the `createAppError` utility in `utils/createAppError.js`.
2. Implement the routes and controllers for registration and login endpoints in `routes/auth.js` and `controllers/authController.js` respectively.
3. Implement the service layer in `services/authService.js` to handle the core business logic for user registration and login. Use the `createAppError` utility to handle errors gracefully.
4. Implement the authentication middleware in `middleware/authHandler.js`.

After implementing the authentication endpoints, you can test the endpoints using Postman. Register a new user and then log in with the same credentials to get a JWT token. Use this token to access a protected route and verify that the authentication is working correctly.

Once the implementation is verified, commit your changes to GitHub with a clear commit message.

Part 2: AI-Assisted Development — Security Enhancements, Unit and Integration Testing, Documentation

Generate Custom Instructions

In the previous week we saw how to manually set up Custom Instructions for GitHub Copilot. We can however let the agent generate the instructions for us based on the codebase and our requirements.

Use `/init` command to generate custom instructions for this project. Use a powerful model like Claude Opus 4.6 which will explore the codebase and generate detailed instructions.

Review the generated instructions and make any necessary adjustments. Save the instructions in .github/copilot-instructions.md file.

Securing the Backend with AI

Security is a critical aspect of backend development. GitHub Copilot can assist you in identifying potential security vulnerabilities in your code and suggesting fixes. You can use the Plan mode to analyze your code for security issues and get recommendations on how to address them.

Start a new chat in **Plan Mode** and use the following prompt to review your backend code for security vulnerabilities:

```
Analyze this codebase and perform a **comprehensive security audit**. The
codebase is built to be a production-grade web application with user
accounts and protected routes.

## Your Objectives

1. Analyze the code for security vulnerabilities and misconfigurations
across the following areas:
    - Authentication & Session Security
    - Authorization & Access Control
    - Input Validation & Injection Risks
    - API & Transport Security
    - Data Protection
    - Dependency & Configuration Issues
2. Order the identified issues in decreasing order of seriousness
(Critical, High, Medium, Low).
3. For each issue, mention the location in the codebase where this occurs
and suggest a fix for the same.
4. Do not generate any code as of now, only help identify the security
vulnerabilities.
```

Review the identified vulnerabilities. You can ask follow-up questions to get more details about a vulnerability.

Identify the top 2-3 critical vulnerabilities that you want to address first. You can continue the conversation in Plan mode to get detailed recommendations on how to fix these vulnerabilities.

```
I want to fix the following critical vulnerabilities:
1. SQL Injection in the /api/posts endpoint
2. Missing authentication on the /api/posts/:id endpoint

Suggest a plan to fix this vulnerability, avoid including code snippets
unless absolutely necessary. I want to understand the steps I need to take
to fix this.
```

You can iterate with the plan to refine it until you have a clear understanding of the steps needed to fix the vulnerability. Once you have a plan, switch to Agent mode and implement the recommended changes.

1. Switch to **Agent Mode** and use the following prompt to implement the fixes:

```
Implement the above plan.
```

Carefully review the code changes made by the agent. Make sure the vulnerabilities have been addressed correctly and that there are no unintended side effects.

Once you are satisfied with the changes, commit them to the repository.

Custom Agent for Generating Tests

Custom agents allow you to create reusable AI assistants that can perform specific tasks. These are specialized versions of the Github Copilot Coding Agents, with user-defined AI personas that tailor the agent's behavior and tools for a specific task.

In this section, you will create a custom agent that can generate unit tests for your API endpoints. This agent can be reused in the future whenever you need to generate tests for new endpoints.

Generating tests by hand can be time-consuming, especially for complex endpoints with multiple scenarios. As the codebase grows, it becomes increasingly difficult to maintain comprehensive test coverage. A custom agent can automate this process, ensuring that you have high-quality tests that cover all necessary scenarios without having to write them manually.

To create a custom agent:

1. Open the agent dropdown in the Chat Panel and select "Configure Custom Agents.."
2. Select "Create New Custom Agent .." in the dropdown options, and then select the location as `.github/agents`.
3. Name your agent `backend-testing-agent`. You will see a template file `backend-testing-agent.agent.md` created in the `.github/agents` directory.
4. Configure the Agent's profile. Add the contents in the `resources\backend-testing-agent.agent.md` into the `backend-testing-agent.agent.md` file you just created. This agent is designed to generate high-quality unit and integration tests for your backend API endpoints.
5. Save the file. Your custom agent is now ready to use.

Now you can use this agent whenever you want to generate tests for your backend code. Simply start a new chat in Agent mode, select the `test-generator` agent, and provide the relevant code files or snippets as input. The agent will analyze the code and generate comprehensive tests based on the defined strategy.

For example, give the following prompt to generate tests for your posts API:

```
Generate tests for the /api/threads endpoint.
```

Review the generated tests and make any necessary adjustments. You can generate tests iteratively, asking the agent to add more test cases or for additional endpoints.

In order to run the generated tests, you may need to set up a test script in your `package.json` file. You can ask the agent to help you with this as well:

```
Help me set up a test script in my package.json to run the generated tests
using Vitest
```

You can run the tests using the following command:

```
npm run test
```

Once satisfied with your code commit it to GitHub.

Documenting the codebase with AI

Documentation is an essential part of software development. It helps other developers understand your code and how to use it. README files are commonly used to provide an overview of the project, while JSDoc comments are used to document specific functions and modules in the code.

Follow the same steps as before to create a new custom agent named `readme-generator`. You can use the `resources\readme-generator.agent.md` file as the profile for this agent. This agent is designed to generate a detailed README file for your project, including sections like project overview, features, installation instructions, usage examples, and more.

Now you can use this agent to generate a README file for your project. Start a new chat in Agent mode, select the `readme-generator` agent, and provide the relevant input (project code, feature list, etc.) to generate the README.

```
Generate a README file for my project. Use the project code and structure
to infer all necessary details.
```

Carefully review the generated README file and make any necessary adjustments. Once you are satisfied with the README, commit it to GitHub with a clear commit message indicating that you have added or updated the project documentation.

This completes the building of the backend of our ThreadHive application. You should now have a complete backend, designed, implemented, tested and documented with AI assistance.